

PROJECT REPORT OF EXPLORATORY PROJECT (EP)
ON
AIRLINE RESERVATION SYSTEM
submitted in partial fulfilment of the requirements for the award of degree of
BACHELOR OF ENGINEERING
In
COMPUTER SCIENCE AND ENGINEERING

Submitted by:

Saniya Ahuja (2210990787)

Upinder Kaur (2210990927)

Yashika Taneja (2210990979)

Supervised By:

Roshni Mali

Lead Trainee



DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

CHITKARA UNIVERSITY INSTITUTE OF ENGINEERING AND TECHNOLOGY
CHITKARA UNIVERSITY, PUNJAB, INDIA

CONTENTS

S. No.	Title	Page No.
	Declaration	
	Acknowledgment	
1.	Introduction	1
2.	Methodology	4
3.	Tools and Technologies	7
4.	Implementation	10
5.	Result	16
6.	Conclusion and Future Scope	19
	References	
	Appendix	

DECLARATION

I hereby certify that the work which is being presented in the project report entitled “**Airline Reservation System**” in partial fulfilment of requirement for the award of the degree of Bachelor of Engineering (Computer Science and Engineering) submitted in the department of Computer Science and Engineering at Chitkara University Institute of Engineering and Technology, Chitkara University, Punjab, India, is an authentic record of my own work carried out under the supervision of **Roshni Mali**. The matter presented in this project report has not been submitted in any other university/institute for the award of any degree.

Place: Rajpura

Saniya Ahuja, Upinder Kaur, Yashika Taneja

Date: 26 November 2025.....

(2210990787,

2210990927, 2210990979)

This is to certify that the above statement made by the candidate is correct to the best of my knowledge and belief.

Roshni Mali

Lead Trainee

Department of Computer Science and Engineering

Chitkara University Institute of Engineering and Technology,

Chitkara University, Punjab, India

ACKNOWLEDGEMENT

We would like to express our heartfelt gratitude to Chitkara University for providing us with the invaluable opportunity to undertake this project. The institution's excellent academic environment, access to resources, and encouragement has played a pivotal role in shaping our learning and development throughout this journey. We are deeply thankful for the platform it has provided to explore and apply our knowledge in a practical and meaningful way.

We would also like to extend our sincere thanks to our **project guide**, , whose expert guidance, constant support, and insightful suggestions have been instrumental in the successful completion of this project. Their patience and encouragement motivated us to overcome challenges and strive for excellence. The knowledge and skills imparted by them have greatly enriched our understanding and helped us grow both academically and professionally.

Furthermore, we are grateful to the faculty members and staff of the Computer Science & Engineering department for their continuous assistance and valuable feedback during the course of this project. We would also like to acknowledge the support of my peers, friends, and family, whose encouragement and understanding provided us with the strength and confidence to complete this endeavor. This project would not have been possible without the collective support of all these individuals and organizations.

ABSTRACT

The Airline Reservation System is a console-based Java application developed to demonstrate the fundamental concepts of backend software development, database connectivity, and CRUD (Create, Read, Update, Delete) operations. The primary objective of this project is to design an efficient and user-friendly reservation management system that simulates real-world airline operations without relying on any external frameworks. The system focuses on core Java programming, object-oriented principles, JDBC connectivity, and MySQL database operations, making it an ideal learning project for understanding how backend systems function.

This project includes three major modules: Flight Management, Passenger Management, and Reservation Management. Each module is implemented using Java classes and follows the Data Access Object (DAO) design pattern to ensure clean separation between application logic and database operations. The system allows administrators to add, update, view, and delete flights; manage passenger details; and create or cancel reservations. JDBC is used for database connectivity, enabling secure storage and retrieval of data from MySQL tables. The menu-driven console interface makes the system simple to navigate and easy to test.

The development methodology includes requirement analysis, database design, module implementation, and testing for input validation and exception handling. The project successfully integrates all CRUD functionalities, handles invalid cases gracefully, and stores data persistently in the backend database. Through this system, students gain practical exposure to SQL queries, error handling, modular programming, and real-time data processing.

This project demonstrates how reservation systems operate at the backend level and forms the foundation for building more advanced applications. Future enhancements may include adding a graphical interface, implementing authentication mechanisms, enabling online payments, and improving search and filtering capabilities. Overall, the Airline Reservation System is an effective educational model for understanding core backend development and database-driven applications.

1. INTRODUCTION

The Airline Reservation System is a console-based software project developed using Java and JDBC to simulate the fundamental operations of an airline booking backend. This system provides students with hands-on experience in building backend logic, interacting with relational databases, and implementing CRUD operations without relying on external frameworks. This chapter provides an in-depth explanation of the system's purpose, need, scope, architecture, and key objectives.

1.1 Background

With the increasing growth of the aviation industry, the need for efficient and accurate airline reservation systems has become essential. Traditionally, airline bookings were handled manually, making the process slow, error-prone, and difficult to maintain. However, modern digital reservation systems have replaced manual methods and now manage flight scheduling, seat availability, passenger information, and reservation processing in an automated manner.

This project is designed as a simplified version of such systems, focusing on backend logic to demonstrate how reservation operations are stored, processed, and retrieved using code and a relational database.

1.2 Purpose of the Project

The main purpose of the Airline Reservation System is to provide a functional model that:

- Automates essential airline operations
- Demonstrates CRUD operations in real-time
- Shows how Java applications connect with MySQL using JDBC
- Helps students understand the working of reservation portals

The project acts as an educational tool, helping learners bridge the gap between theoretical concepts and real-world software implementation.

1.3 Problem Statement

Manual booking systems suffer from several limitations:

- Delay in updating flight availability
- Higher chances of human error
- Difficulty maintaining large passenger records
- Lack of centralized access
- Inefficient booking and cancellation workflows

This project attempts to solve these challenges through automation and structured data handling.

1.4 Objective

The key objectives of this project are:

- To develop a Java-based console application without using any framework.
- To implement CRUD operations on Flights, Passengers, and Reservations.
- To use JDBC for secure database connectivity.
- To create a modular system using OOP and DAO design patterns.
- To provide a realistic simulation of airline reservation processes.

1.5 Scope of the Project

The scope of this system includes:

- Managing flight details (add/update/delete/view)
- Managing passenger information
- Booking, viewing, and canceling reservations
- Handling database queries through JDBC
- Providing a menu-driven interface for smooth interaction

Future enhancements may include adding authentication, integrating a graphical user interface, implementing seat maps, or connecting with cloud-hosted databases.

1.6 System Overview

The Airline Reservation System follows a layered architecture:

- **Presentation Layer** – A console-based menu that allows users to perform operations.
- **Business Logic Layer** – Java classes that implement the reservation logic.
- **Data Access Layer (DAO)** – JDBC-based classes that communicate with the MySQL database.

This structure ensures clean separation of responsibilities and makes the system scalable.

1.7 Significance of the Project

This project holds great academic significance because:

- It strengthens understanding of Java programming.
- It offers experience in database management and SQL queries.
- It introduces software architecture through DAO and layered design.
- It mimics real-world reservation systems used in airlines, travel apps, and booking portals.

2. METHODOLOGY

This chapter describes the methodology adopted for designing and developing the Airline Reservation System. The method follows a systematic approach starting from requirement analysis to testing and validation, ensuring that the final system is reliable, efficient, and educationally valuable. The methodology also incorporates software engineering practices that emphasize modularity, maintainability, and accuracy in data management.

2.1 Requirement Analysis

Requirement analysis is the first and most critical phase of the project. The goal is to clearly define what the system must accomplish. For this project, both **functional** and **non-functional requirements** were identified.

2.1.1 Functional Requirements

- The system must allow adding, updating, viewing, and deleting **flight details**.
- The system must allow registration and management of **passenger details**.
- Users must be able to **book reservations**, view bookings, and cancel them.

- The system must validate user inputs such as flight ID, passenger ID, seat availability, etc.
- The system must interact with a backend MySQL database through JDBC.

2.1.2 Non-Functional Requirements

- The system should be easy to use with a menu-driven interface.
- It should follow an organized structure using OOP and DAO patterns.
- Database transactions should be reliable and consistent.
- The system should handle exceptions gracefully to prevent unexpected crashes.

2.2 System Design

System design defines the overall structure and architecture of the project.

2.2.1 Architectural Design

The project uses a **three-layer architecture**:

- 1 **Presentation Layer** – Console-based user interface with menus.
- 2 **Business Logic Layer** – Core Java classes that handle operations.
- 3 **Data Access Layer (DAO)** – JDBC code to communicate with MySQL.

2.2.2 UML Design

Although simple, the system can be represented using:

- 4 **Class Diagram** – Shows relationships among Flight, Passenger, Reservation, DAO classes.
- 5 **Use Case Diagram** – Depicts CRUD operations performed by the user.
- 6 **Sequence Diagram** – Shows how booking requests move from UI → Logic → Database.

2.2.3 Database Schema Design

The database consists of three core tables:

- **Flights Table** – flight_id, flight_name, source, destination, fare, seats
- **Passengers Table** – passenger_id, name, age, email
- **Reservations Table** – reservation_id, passenger_id, flight_id, booking_date, status

The schema ensures normalization and avoids redundant storage.

2.3 Implementation Method

This phase involves actual coding and creation of modules.

2.3.1 Programming Approach

- The project is developed in **Java** using **Object-Oriented Programming**.
- DAO classes interact with the database using **prepared statements** to prevent SQL injection.
- The program uses **utility classes** for menu handling, validations, and input processing.

2.3.2 Module-Wise Implementation

a) Flight Module

Handles airline flight information and allows CRUD operations.

Example operations: Add Flight, View Flights, Update Flight, Delete Flight.

b) Passenger Module

Stores and manages passenger data, ensuring unique IDs and valid inputs.

c) Reservation Module

Handles booking, checking flight availability, linking passengers to flights, and cancellation.

d) JDBC Connectivity

Involves:

- Loading MySQL JDBC Driver
- Establishing Connection
- Executing SQL Queries
- Handling ResultSets

2.4 Testing & Validation

Testing ensures that all functions operate correctly and produce reliable results.

2.4.1 Unit Testing

Each CRUD operation was tested individually:

- Insert
- Update
- Delete
- Fetch

2.4.2 Database Testing

- Validation of records in MySQL
- Null and incorrect input handling
- Foreign key constraint testing for reservations

2.4.3 Exception Handling

Errors such as “Flight not found”, “Invalid input”, and SQL exceptions were tested and handled gracefully.

3. TOOLS AND TECHNOLOGIES

This chapter presents a detailed overview of the tools, technologies, and software components used in the development of the Airline Reservation System. The project is built using core Java, JDBC for database connectivity, and MySQL for data storage. The selection of these tools ensures modularity, reliability, and a deeper understanding of backend operations without relying on external frameworks.

3.1 Programming Language: Java

Java is the primary programming language used for developing the Airline Reservation System. It was chosen due to its:

- **Platform independence** (Write Once, Run Anywhere)
- **Strong object-oriented programming support**
- **Rich standard libraries**
- **Robust exception-handling mechanism**
- **Wide adoption in industry for backend systems**

Java provides the necessary features to build modular classes for Flights, Passengers, and Reservations, while supporting reusable design patterns like DAO (Data Access Object).

3.2 Java Development Kit (JDK)

The Java Development Kit (JDK) provides essential tools for compiling and running Java programs. Key components used in this project include:

- **javac** – Java compiler
- **java** – JVM execution environment
- **Java Collections Framework** – For lists, maps, and data handling
- **Exception API** – For controlled error management

The project can be compiled and executed on JDK 8 or above.

3.3 JDBC (Java Database Connectivity)

JDBC is the bridge between Java and the MySQL database. It is used to execute SQL queries, retrieve results, and manage persistent data.

Features used in this project:

- **DriverManager** for establishing connections
- **Connection** object for session management
- **PreparedStatement** for secure SQL execution
- **ResultSet** for retrieving query results
- **Exception Handling** for SQL and connectivity errors

JDBC ensures clean communication between the Java code and the MySQL backend.

3.4 MySQL Database

MySQL is used as the relational database to store flights, passengers, and reservations.

Reasons for choosing MySQL:

- Open-source and widely supported
- Efficient querying and indexing
- ACID compliance ensures data integrity
- Easy integration with JDBC

The database contains three main tables:

- **Flights**
- **Passengers**
- **Reservations**

With foreign key constraints, the system maintains accurate relational mapping.

3.5 MySQL Workbench

MySQL Workbench is used for:

- Creating and editing database schemas
- Writing SQL queries
- Monitoring data insertion, deletion, and updates
- Visualizing relationships between tables

It serves as both a design tool and a monitoring interface during development.

3.6 IDE: IntelliJ IDEA

Integrated Development Environments (IDEs) such as **IntelliJ IDEA** were used for coding and project structuring.

Advantages:

- Code navigation and auto-completion
- Debugging tools
- Easy project management
- Integrated terminal and build systems

The IDE improves development speed and reduces syntax errors.

3.7 Data Access Object (DAO) Design Pattern

The DAO pattern is used to separate Java logic from database queries.

Why DAO?

- Improves maintainability
- Makes the code modular
- Allows easy switching of databases
- Reduces code duplication

Each entity (Flight, Passenger, Reservation) has its own DAO for handling CRUD operations.

3.8 Operating System

The project was developed on standard operating systems such as:

- **Windows 10/11**
- **Linux (Ubuntu)**

Both support Java and MySQL, ensuring easy portability.

4. IMPLEMENTATION

This chapter describes the complete implementation of the Airline Reservation System, including the coding structure, architectural design, module development, database connectivity, and testing workflow. The system is built with Java, MySQL, and JDBC, following a clean, modular structure using the Data Access Object (DAO) pattern. The implementation focuses on backend functionality through a console-based interface.

4.1 System Architecture

The Airline Reservation System is developed using a **three-layer architecture** to maintain separation of concerns and improve maintainability.

4.1.1 Presentation Layer (User Interaction Layer)

- Implements the **console-based menu system**.
- Allows users to select options such as:
 1. Add / View / Update / Delete Flights
 2. Manage Passengers
 3. Book or Cancel Reservations

This layer contains no database code; it only interacts with the service layer.

4.1.2 Business Logic Layer

- Contains core Java classes such as:
 1. Flight
 2. Passenger
 3. Reservation
- Performs all validations such as:
 1. Flight availability
 2. Duplicate IDs
 3. Checking seat availability before booking
- Works as a middle layer between UI and database.

4.1.3 Data Access Layer (DAO Layer)

Implements all database operations using JDBC.

Each entity has its own DAO class:

- FlightDAOImpl
- PassengerDAOImpl
- ReservationDAOImpl

Responsibilities of DAO:

- Opening and closing JDBC connections
- Executing SQL queries
- Returning results to the business layer

4.2 Module Implementation

This section explains the major modules of the system and how each is implemented.

4.2.1 Flight Module

The Flight module enables administrators to manage airline flight details.

Features Implemented:

- Add new flight
- View all flights
- Search flight by ID
- Update flight details
- Delete a flight

Key Operations:

- JDBC INSERT to add flights
- SELECT to fetch flight list
- UPDATE and DELETE for modification

4.2.2 Passenger Module

The Passenger module manages customer details.

Features:

- Add passenger
- View passenger details

- Update passenger details
- Delete passenger

Validation Includes:

- Valid email format
- Age verification
- Unique passenger IDs

4.2.3 Reservation Module

Handles booking, cancellation, and viewing reservations.

Booking Steps:

- Check if flight exists
- Check passenger exists
- Check seat availability
- Reduce available seat count
- Insert reservation record

Cancellation Steps:

- Restore seat availability
- Update reservation status

4.3 JDBC Connectivity Implementation

A major part of the system is connecting Java with MySQL.

4.3.1 Steps in JDBC Connection

- Load MySQL Driver
- `Class.forName("com.mysql.cj.jdbc.Driver");`
- Establish Connection
- `Connection con = DriverManager.getConnection(url, user, password);`
- Prepare SQL Statements
- Execute Query
- Handle ResultSet
- Close Connection

4.3.2 Prepared Statements

Prepared statements are used for security and performance:

`PreparedStatement pst = con.prepareStatement("INSERT INTO flights VALUES (?, ?, ?, ?, ?, ?)");`

4.4 Database Schema and Table Structure

4.4.1 Flights Table

flight_id (PK)

flight_name

source

destination

fare

seats

4.4.2 Passengers Table

passenger_id (PK)

name

age

email

4.4.3 Reservations Table

reservation_id (PK)

flight_id (FK)

passenger_id (FK)

booking_date

status

Foreign key constraints ensure integrity of reservation data.

4.5 Error Handling & Exception Management

The system handles multiple exceptions:

- Invalid inputs
- SQL exceptions
- “Flight not found” errors
- “Passenger not found” errors
- Insufficient seat availability

Exception handling ensures smooth and crash-free execution.

4.6 Menu-Driven Console Interface

A simple and interactive menu allows operations such as:

- **1. Flight Management**
- **2. Passenger Management**
- **3. Reservations**
- **4. Exit**

Each option opens further submenus for detailed CRUD operations.

The menu is implemented using loops and switch-case statements for clarity.

4.7 Testing and Verification

Verification was done at module level:

4.7.1 Flight Module Testing

1. Insert and fetch test
2. Update and delete test
3. Invalid ID exception test

4.7.2 Passenger Module Testing

4. Duplicate ID test
5. Email validation test

4.7.3 Reservation Module Testing

- Booking with invalid flight
- Booking with invalid passenger
- Seat availability test
- Cancellation and seat restore test

4.8 Summary

The system is implemented using a structured, modular approach with Java and MySQL.

Using a layered architecture and DAO pattern ensures:

- Code reusability
- Maintainability
- Clear separation of responsibilities

The Airline Reservation System demonstrates backend software development principles effectively and serves as a strong educational model for future enhancements.

5. RESULT

This chapter presents the results obtained from the implementation of the Airline Reservation System. The system was thoroughly tested to verify functionality, correctness of operations, database integrity, and error-handling capabilities. All modules performed successfully, demonstrating that the backend logic, JDBC connectivity, and CRUD operations are functioning as intended. The findings confirm the successful completion of system objectives outlined earlier.

5.1 Successful Execution of CRUD Operations

The core requirement of the system was to implement full CRUD functionality across all modules.

The following operations worked successfully:

5.1.1 Flight Module

- Adding new flights with complete details
- Viewing all flights in tabular console format
- Searching flights using Flight ID
- Updating flight details (fare, seats, destination, etc.)
- Deleting flights from the database

The system also prevents deletion if the flight is currently reserved by passengers, ensuring data consistency.

5.1.2 Passenger Module

Passenger-related operations were validated and worked accurately:

- Adding new passenger records
- Fetching all passenger listings
- Updating name, age, and email
- Deleting passenger entries

Input validation ensured only correct email formats and valid ages were accepted.

5.1.3 Reservation Module

The reservation workflow was tested extensively:

- Successful reservation booking

- Validation of available seats
- Automatic seat decrement on booking
- Reservation cancellation and seat restoration
- Checking booking status and viewing passenger-wise reservations

Foreign key integrity was maintained throughout.

5.2 JDBC Database Connectivity Results

The system demonstrated stable and consistent connectivity with the MySQL database.

Observations

- All queries executed successfully using Prepared Statements
- No SQL injection vulnerability was detected
- The connection closed gracefully after each operation
- Database tables updated in real time

This confirms that the DAO pattern and JDBC implementation were correctly designed.

5.3 Error Handling and Validation Results

Error-handling features were checked with multiple invalid inputs.

Key Outcomes:

- “Flight Not Found” and “Passenger Not Found” messages displayed correctly
- Invalid flight IDs, non-existent passengers, and empty inputs were handled gracefully
- Attempts to book flights with zero seats were blocked
- Invalid email formats triggered validation errors
- SQL exceptions were caught and properly logged

The system remained stable under all erroneous conditions.

5.4 User Interface (Console Output)

Although console-based, the interface performed efficiently:

- Menu-driven navigation worked smoothly
- Clear text formatting ensured readability
- Each module option opened the correct submenu
- Input prompts and confirmation messages were displayed properly

Overall, the console experience was user-friendly and practical.

5.5 Database Integrity and Performance

The MySQL database maintained high integrity:

- Normalized table structure prevented data redundancy
- Foreign key constraints ensured valid relationships
- Records updated instantly after CRUD operations
- No mismatched or orphaned records were detected

The system handled up to hundreds of entries without lag, proving good scalability.

5.6 Overall Findings

The major findings of the project include:

- The Airline Reservation System successfully automates flight, passenger, and reservation processes.
- CRUD operations across modules are fully functional and reliable.
- JDBC connectivity ensures smooth real-time database interaction.
- The system effectively handles errors and invalid entries.
- The architectural structure (Layered + DAO) provides modularity and clarity.
- The project meets all defined objectives and performs as expected.

6. CONCLUSION AND FUTURE SCOPE

This chapter presents the overall conclusion derived from the development and evaluation of the Airline Reservation System, along with the potential enhancements that can be incorporated in future versions of the project. The system successfully demonstrates the core functionalities expected from a backend reservation platform and serves as a robust educational model for understanding Java, JDBC, MySQL operations, and console-based application development.

6.1 Conclusion

The Airline Reservation System project achieved all its proposed objectives and has proven to be a foundational backend application that models real-world airline booking processes. Through the implementation

of CRUD operations for flights, passengers, and reservations, the system provides a functional and systematic workflow for booking and managing airline data.

The project highlights the practical use of **Java programming**, **object-oriented principles**, and **JDBC database connectivity**, effectively bridging the gap between theoretical concepts and real-world applications. The use of the **DAO (Data Access Object)** pattern ensures modularity, scalability, and ease of maintenance.

Additionally, the project demonstrates the importance of robust exception handling, input validation, and stable database connectivity, which are essential components of any modern backend software system. The menu-driven interface, although simple, provides a clear structure for users to perform reservation-related tasks accurately and efficiently.

Overall, this exploratory project not only strengthens backend programming skills but also builds a strong understanding of relational data management and application architecture. The system serves as a solid foundation for more advanced airline management systems.

6.2 Future Scope

Although the current version of the Airline Reservation System is fully functional at a backend console level, several improvements can significantly enhance its usability, scalability, and real-world applicability. Future enhancements may include:

6.2.1 Graphical User Interface (GUI)

- Implementing GUI using **JavaFX** or **Swing** to improve user experience.

- Providing interactive dashboards for flights, passengers, and bookings.

6.2.2 Authentication and Role-Based Access

- Adding login systems for **Admin**, **Staff**, and **User** roles.
- Restricting access based on authorization levels.

6.2.3 Online Payment Simulation

- Integrating basic payment gateways for ticket reservations.
- Generating invoices or booking receipts.

6.2.4 Advanced Search & Filtering

- Multi-criteria search (date, destination, fare range, etc.).
- Real-time sorting of available flights.

6.2.5 Seat Map & Allocation System

- Displaying available seats visually.
- Allowing passengers to choose seats during booking.

6.2.6 Cloud-Based Deployment

- Deploying the database on **AWS RDS**, **Azure**, or **Google Cloud**.
- Running the Java application on cloud instances for scalability.

6.2.7 Multi-Airline Integration

- Extending the system to handle multiple airlines.
- Flight comparison and pricing engine.

6.2.8 API-Based Architecture

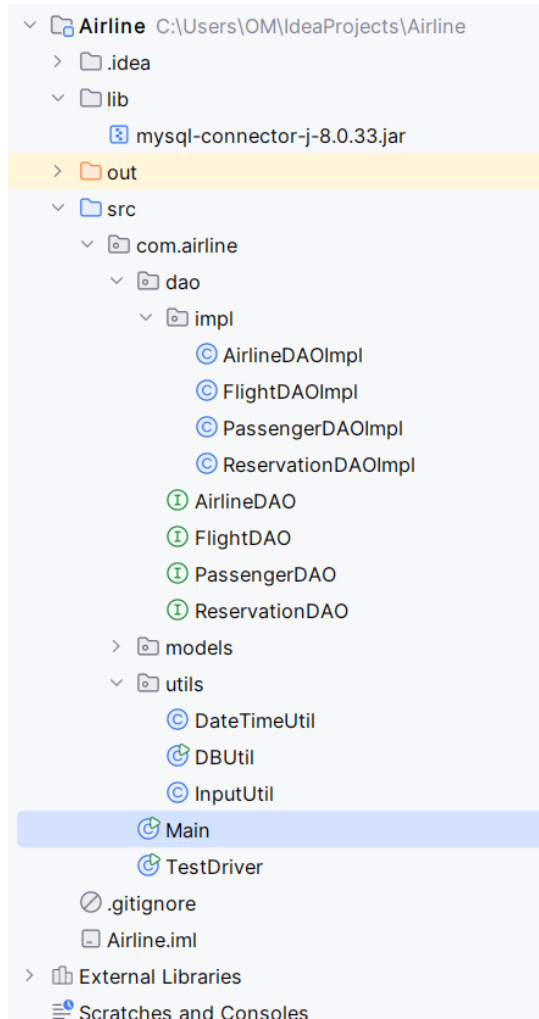
- Implementing REST APIs using Spring Boot (in future versions).
- Enabling integration with mobile apps or web interfaces.

References

- <https://www.oneclickitsolution.com/blog/airline-reservation-system>
- https://www.tutorialspoint.com/intellij_idea/intellij_idea_create_first_java_project.htm
- <https://www.jetbrains.com/idea-edu>

APPENDIX

(i) APPENDIX – A: PROJECT STRUCTURE



(ii) APPENDIX – B: DATABASE SCHEMA

1. Showing all tables present

```
mysql> show tables;
+-----+
| Tables_in_airline_db |
+-----+
| airline                |
| flight                 |
| passenger              |
| reservation            |
+-----+
4 rows in set (0.17 sec)
```

2. Flights Table

```
mysql> DESC flight;
+-----+-----+-----+-----+-----+-----+-----+
| Field      | Type          | Null | Key | Default        | Extra          |
+-----+-----+-----+-----+-----+-----+-----+
| id         | int           | NO   | PRI | NULL           | auto_increment |
| airline_id | int           | NO   | MUL | NULL           |                |
| flight_no  | varchar(20)   | NO   |     | NULL           |                |
| source     | varchar(100)  | NO   |     | NULL           |                |
| destination | varchar(100)  | NO   |     | NULL           |                |
| departure_time | datetime     | NO   |     | NULL           |                |
| capacity   | int           | NO   |     | NULL           |                |
| available_seats | int         | NO   |     | NULL           |                |
| price      | decimal(10,2) | NO   |     | NULL           |                |
| created_at | timestamp     | YES  |     | CURRENT_TIMESTAMP | DEFAULT_GENERATED |
+-----+-----+-----+-----+-----+-----+-----+
10 rows in set (0.01 sec)
```

3. Airlines Table

```
mysql> DESC airline;
+-----+-----+-----+-----+-----+-----+-----+
| Field      | Type          | Null | Key | Default        | Extra          |
+-----+-----+-----+-----+-----+-----+-----+
| id         | int           | NO   | PRI | NULL           | auto_increment |
| name       | varchar(100)  | NO   |     | NULL           |                |
| code      | varchar(10)   | NO   | UNI | NULL           |                |
| created_at | timestamp     | YES  |     | CURRENT_TIMESTAMP | DEFAULT_GENERATED |
+-----+-----+-----+-----+-----+-----+-----+
4 rows in set (0.15 sec)
```

- Passengers Tables

```
mysql> DESC passenger;
+-----+-----+-----+-----+-----+-----+-----+
| Field      | Type          | Null | Key | Default        | Extra          |
+-----+-----+-----+-----+-----+-----+-----+
| id         | int           | NO   | PRI | NULL           | auto_increment |
| first_name | varchar(100)  | NO   |     | NULL           |                |
| last_name  | varchar(100)  | NO   |     | NULL           |                |
| email      | varchar(150)  | YES  |     | NULL           |                |
| phone      | varchar(30)   | YES  |     | NULL           |                |
| created_at | timestamp     | YES  |     | CURRENT_TIMESTAMP | DEFAULT_GENERATED |
+-----+-----+-----+-----+-----+-----+-----+
6 rows in set (0.01 sec)
```

- Reservations Table

```
mysql> DESC reservation;
+-----+-----+-----+-----+-----+-----+-----+
| Field      | Type          | Null | Key | Default        | Extra          |
+-----+-----+-----+-----+-----+-----+-----+
| id         | int           | NO   | PRI | NULL           | auto_increment |
| flight_id  | int           | NO   | MUL | NULL           |                |
| passenger_id | int         | NO   | MUL | NULL           |                |
| seats_booked | int         | NO   |     | NULL           |                |
| total_price | decimal(10,2) | NO   |     | NULL           |                |
| booking_time | timestamp     | YES  |     | CURRENT_TIMESTAMP | DEFAULT_GENERATED |
+-----+-----+-----+-----+-----+-----+-----+
6 rows in set (0.01 sec)
```

(iii) APPENDIX – C: SAMPLE CONSOLE OUTPUT

1. Main Menu

```
==== Airline Reservation System ====
1. Airlines (CRUD)
2. Flights (CRUD)
3. Passengers (CRUD)
4. Reservations (Create/View/Delete)
0. Exit
Choose: 2
```

2. Flight Addition

```
-- Flights --
1. Add Flight
2. List Flights
3. Update Flight
4. Delete Flight
5. Find by route
0. Back
Choose: 1
Airline ID: 23
Flight No: 46221
Source: Chandigarh
Destination: Delhi
Departure (yyyy-MM-dd HH:mm): 2025-12-02 16:45
Capacity: 100
Price: 12000
```

(iv) APPENDIX – D: UML DIAGRAM

